

Name = Waheed Haider Rizvi

Roll no = 202501101400060

Branch = CSE CS

Subject = OOPs in java

**Question:**

WAP to insert 3 numbers from the keyboard and find a greater number among 3 numbers.

```
import java.util.Scanner;
```

```
class GreaterNumber {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        System.out.print("Enter first number: ");
```

```
        int a = sc.nextInt();
```

```
System.out.print("Enter second number: ");
```

```
int b = sc.nextInt();
```

```
System.out.print("Enter third number: ");
```

```
int c = sc.nextInt();
```

```
int greater;
```

```
if (a >= b && a >= c) {
```

```
    greater = a;
```

```
} else if (b >= a && b >= c) {
```

```
    greater = b;
```

```
} else {
```

```
    greater = c;
```

```
}
```

```
    System.out.println("Greater number = " +  
greater);
```

```
}
```

```
}
```

### **Question:**

WAP to count the total number of odd numbers between 1-100, and display the sum of them.

```
class OddNumbers {
```

```
    public static void main(String[] args) {
```

```
        int count = 0;
```

```
        int sum = 0;
```

```
        for (int i = 1; i <= 100; i++) {
```

```
        if (i % 2 != 0) {  
            count++;  
            sum += i;  
        }  
    }
```

```
        System.out.println("Total odd numbers = "  
+ count);
```

```
        System.out.println("Sum of odd numbers = "  
+ sum);  
    }  
}
```

### **Question:**

WAP to find largest and smallest numbers in an array

```
import java.util.Scanner;
```

```
class ArrayMinMax {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
  
        System.out.print("Enter size of array: ");  
        int n = sc.nextInt();  
  
        int[] arr = new int[n];  
  
        System.out.println("Enter array elements:");  
  
        for (int i = 0; i < n; i++) {  
            arr[i] = sc.nextInt();  
        }  
    }  
}
```

```
int largest = arr[0];  
int smallest = arr[0];
```

```
for (int i = 1; i < n; i++) {  
    if (arr[i] > largest) {  
        largest = arr[i];  
    }  
}
```

```
    if (arr[i] < smallest) {  
        smallest = arr[i];  
    }  
}
```

```
System.out.println("Largest = " + largest);
```

```
        System.out.println("Smallest = " + smallest);
    }
}
```

### **Question:**

WAP to count the number of characters in a given string, reverse the string and check whether it is palindrome or not.

```
import java.util.Scanner;
```

```
class StringOperations {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a string: ");
        String str = sc.nextLine();
```

```
int count = str.length();
```

```
String reverse = "";
```

```
for (int i = str.length() - 1; i >= 0; i--) {  
    reverse = reverse + str.charAt(i);  
}
```

```
System.out.println("Number of characters  
= " + count);
```

```
System.out.println("Reversed string = " +  
reverse);
```

```
if (str.equalsIgnoreCase(reverse)) {  
    System.out.println("Palindrome = True");
```

```
    } else {  
        System.out.println("Palindrome = False");  
    }  
}  
}
```

### **Question:**

WAP to find out the sum of command line arguments.

```
class CommandLineSum {  
    public static void main(String[] args) {  
        int sum = 0;  
  
        for (int i = 0; i < args.length; i++) {  
            sum = sum + Integer.parseInt(args[i]);  
        }  
    }  
}
```

```
        System.out.println("Sum = " + sum);  
    }  
}
```

**Question:**

WAP to create a class SimpleObject and display messages by using the constructor of this class.

```
class SimpleObject {
```

```
    SimpleObject() {  
        System.out.println("Constructor is called.");  
        System.out.println("Welcome to Java  
Programming.");  
    }
```

```
public static void main(String[] args) {  
    SimpleObject obj = new SimpleObject();  
}  
}
```

**Question:**

WAP to create a class Number with only one private instance variable as a double primitive type. Include methods isZero(), isPositive(), isNegative(), isOdd(), isEven(), isPrime(), isAmstrong(). All methods should return boolean.

```
import java.util.Scanner;
```

```
class Number {
```

```
private double num;
```

```
Number(double num) {  
    this.num = num;  
}
```

```
boolean isZero() {  
    return num == 0;  
}
```

```
boolean isPositive() {  
    return num > 0;  
}
```

```
boolean isNegative() {
```

```
    return num < 0;  
}
```

```
boolean isOdd() {  
    return num % 2 != 0;  
}
```

```
boolean isEven() {  
    return num % 2 == 0;  
}
```

```
boolean isPrime() {  
  
    if (num < 2 || num != (int) num) {  
        return false;
```

```
}
```

```
int n = (int) num;
```

```
for (int i = 2; i < n; i++) {
```

```
    if (n % i == 0) {
```

```
        return false;
```

```
    }
```

```
}
```

```
return true;
```

```
}
```

```
boolean isAmstrong() {
```

```
if (num < 0 || num != (int) num) {  
    return false;  
}
```

```
int n = (int) num;  
int original = n;  
int digits = String.valueOf(n).length();  
int sum = 0;
```

```
while (n > 0) {  
    int digit = n % 10;  
    sum += Math.pow(digit, digits);  
    n = n / 10;  
}
```

```
    return sum == original;  
}
```

```
public static void main(String[] args) {
```

```
    Scanner sc = new Scanner(System.in);
```

```
    System.out.print("Enter a number: ");
```

```
    double value = sc.nextDouble();
```

```
    Number n = new Number(value);
```

```
    System.out.println("Zero = " + n.isZero());
```

```
    System.out.println("Positive = " +  
n.isPositive());
```

```
        System.out.println("Negative = " +  
n.isNegative());  
  
        System.out.println("Odd = " + n.isOdd());  
  
        System.out.println("Even = " + n.isEven());  
  
        System.out.println("Prime = " + n.isPrime());  
  
        System.out.println("Armstrong = " +  
n.isArmstrong());  
  
    }  
}
```

### **Question:**

WAP to create a Room class. The attributes are roomno, roomtype, roomarea and ACmachine. Use member functions setData() and displayData().

```
import java.util.Scanner;
```

```
class Room {
```

```
    int roomno;
```

```
    String roomtype;
```

```
    double roomarea;
```

```
    boolean ACmachine;
```

```
    void setData() {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        System.out.print("Enter room number: ");
```

```
        roomno = sc.nextInt();
```

```
        sc.nextLine();
```

```
System.out.print("Enter room type: ");  
roomtype = sc.nextLine();
```

```
System.out.print("Enter room area: ");  
roomarea = sc.nextDouble();
```

```
System.out.print("Does room have AC?  
(true/false): ");
```

```
ACmachine = sc.nextBoolean();  
}
```

```
void displayData() {
```

```
System.out.println("\nRoom Details");
```

```
        System.out.println("Room Number = " +  
roomno);
```

```
        System.out.println("Room Type = " +  
roomtype);
```

```
        System.out.println("Room Area = " +  
roomarea);
```

```
        System.out.println("AC Machine = " +  
ACmachine);
```

```
    }
```

```
public static void main(String[] args) {
```

```
    Room r = new Room();
```

```
    r.setData();
```

```
    r.displayData();
```

```
}  
  
}
```

### **Question:**

WAP to create a class named Shape and create three subclasses Circle, Triangle and Square.

Each class has two member functions draw() and erase(). Implement using polymorphism.

```
class Shape {
```

```
    void draw() {
```

```
        System.out.println("Drawing Shape");
```

```
    }
```

```
    void erase() {
```

```
        System.out.println("Erasing Shape");
```

```
    }
```

```
}
```

```
class Circle extends Shape {
```

```
    void draw() {
```

```
        System.out.println("Drawing Circle");
```

```
    }
```

```
    void erase() {
```

```
        System.out.println("Erasing Circle");
```

```
    }
```

```
}
```

```
class Triangle extends Shape {
```

```
void draw() {  
    System.out.println("Drawing Triangle");  
}
```

```
void erase() {  
    System.out.println("Erasing Triangle");  
}  
}
```

```
class Square extends Shape {
```

```
    void draw() {  
        System.out.println("Drawing Square");  
    }
```

```
void erase() {  
    System.out.println("Erasing Square");  
}  
}
```

```
class ShapeDemo {  
  
    public static void main(String[] args) {  
  
        Shape s;  
  
        s = new Circle();  
        s.draw();  
        s.erase();  
    }  
}
```

```
s = new Triangle();  
s.draw();  
s.erase();  
  
s = new Square();  
s.draw();  
s.erase();  
}  
}
```

**Question:**

WAP to create a class Employee with a method calculateSalary(). Create two subclasses Manager and Programmer. Override the method in each subclass.

```
class Employee {
```

```
double calculateSalary() {  
    return 0;  
}  
}
```

```
class Manager extends Employee {
```

```
    double calculateSalary() {  
        return 80000;  
    }  
}
```

```
class Programmer extends Employee {
```

```
    double calculateSalary() {
```

```
        return 60000;
    }
}
```

```
class EmployeeDemo {
```

```
    public static void main(String[] args) {
```

```
        Employee e;
```

```
        e = new Manager();
```

```
        System.out.println("Manager Salary = " +
            e.calculateSalary());
```

```
        e = new Programmer();
```

```
        System.out.println("Programmer Salary = "
+ e.calculateSalary());
    }
}
```

### **Question:**

WAP to implement the concept of method overloading, method overriding and abstract class.

```
abstract class Animal {

    abstract void sound();

    void eat() {
        System.out.println("Animal eats food.");
    }
}
```

```
class Dog extends Animal {
```

```
    // Method overriding
```

```
    void sound() {
```

```
        System.out.println("Dog barks.");
```

```
    }
```

```
    // Method overloading
```

```
    void display() {
```

```
        System.out.println("Dog");
```

```
    }
```

```
    void display(String name) {
```

```
        System.out.println("Dog name = " + name);
```

```
}  
}
```

```
class OOPDemo {
```

```
    public static void main(String[] args) {
```

```
        Dog d = new Dog();
```

```
        // Abstract class method implemented by  
        subclass
```

```
        d.sound();
```

```
        d.eat();
```

```
// Method overloading  
d.display();  
d.display("Tommy");  
}  
}
```

Question.

WAP to create a class Employee having:

- private int empNo
- private String empName
- private int empBasic
- Parameterized constructor
- Getter methods for all variables

Create WriteEmployee with a main method.

Take employee details from the user, store them in emp.txt, read the details and display them.

```
import java.io.*;
```

```
import java.util.Scanner;
```

```
class Employee {
```

```
    private int empNo;
```

```
    private String empName;
```

```
    private int empBasic;
```

```
    Employee(int empNo, String empName, int  
empBasic) {
```

```
        this.empNo = empNo;
```

```
        this.empName = empName;
```

```
        this.empBasic = empBasic;
```

```
    }
```

```
int getEmpNo() {  
    return empNo;  
}
```

```
String getEmpName() {  
    return empName;  
}
```

```
int getEmpBasic() {  
    return empBasic;  
}  
}
```

```
public class WriteEmployee {
```

```
public static void main(String[] args) {  
  
    Scanner sc = new Scanner(System.in);  
  
    System.out.print("Enter Employee Number:  
");  
  
    int empNo = sc.nextInt();  
  
    sc.nextLine();  
  
    System.out.print("Enter Employee Name: ");  
    String empName = sc.nextLine();  
  
    System.out.print("Enter Basic Salary: ");
```

```
int empBasic = sc.nextInt();
```

```
Employee emp = new Employee(empNo,  
empName, empBasic);
```

```
try {
```

```
    FileWriter fw = new FileWriter("emp.txt");
```

```
        fw.write("Employee Number: " +  
emp.getEmpNo() + "\n");
```

```
        fw.write("Employee Name: " +  
emp.getEmpName() + "\n");
```

```
        fw.write("Basic Salary: " +  
emp.getEmpBasic() + "\n");
```

```
    fw.close();
```

```
        System.out.println("\nEmployee details  
written to file.");
```

```
        BufferedReader br = new  
        BufferedReader(  
            new FileReader("emp.txt"));
```

```
        String line;
```

```
        System.out.println("\nEmployee  
Details:");
```

```
        while ((line = br.readLine()) != null) {  
            System.out.println(line);  
        }
```

```
        br.close();

    } catch (IOException e) {

        System.out.println("File error: " +
e.getMessage());

    }

}

}
```

### **Question:**

WAP to create a text file in the path C:/Java/abc.txt and check whether that file exists or not. Use exists(), isDirectory(), isFile(), getName() and getAbsolutePath().

```
import java.io.File;
```

```
import java.io.IOException;
```

```
class FileDemo {  
  
    public static void main(String[] args) {  
  
        File file = new File("C:/Java/abc.txt");  
  
        try {  
  
            if (file.createNewFile()) {  
                System.out.println("File created  
successfully.");  
            } else {  
                System.out.println("File already  
exists.");  
            }  
        }  
    }  
}
```

```
        System.out.println("Exists = " +  
file.exists());
```

```
        System.out.println("Is Directory = " +  
file.isDirectory());
```

```
        System.out.println("Is File = " +  
file.isFile());
```

```
        System.out.println("File Name = " +  
file.getName());
```

```
        System.out.println("Absolute Path = " +  
file.getAbsolutePath());
```

```
    } catch (IOException e) {  
        System.out.println("Error: " +  
e.getMessage());  
    }
```

```
}  
  
}
```

### **Question:**

WAP to implement three classes: Storage, Counter and Printer.

- Storage stores an integer.
- Counter creates a thread and counts from 0, 1, 2, 3... and stores each value in Storage.
- Printer creates a thread that reads the value from Storage and prints it.

```
class Storage {
```

```
    private int value;
```

```
    synchronized void setValue(int value) {
```

```
    this.value = value;  
}
```

```
    synchronized int getValue() {  
        return value;  
    }  
}
```

```
class Counter extends Thread {
```

```
    Storage storage;
```

```
    Counter(Storage storage) {  
        this.storage = storage;  
    }
```

```
public void run() {  
  
    for (int i = 0; i <= 10; i++) {  
  
        storage.setValue(i);  
  
        try {  
            Thread.sleep(100);  
        } catch (InterruptedException e) {  
            System.out.println(e);  
        }  
    }  
}
```

```
class Printer extends Thread {
```

```
    Storage storage;
```

```
    Printer(Storage storage) {
```

```
        this.storage = storage;
```

```
    }
```

```
    public void run() {
```

```
        for (int i = 0; i <= 10; i++) {
```

```
            System.out.println("Value = " +  
storage.getValue());
```

```
    try {  
        Thread.sleep(100);  
    } catch (InterruptedException e) {  
        System.out.println(e);  
    }  
}  
}  
}
```

```
class ThreadStorageDemo {
```

```
    public static void main(String[] args) {
```

```
        Storage storage = new Storage();
```

```
Counter counter = new Counter(storage);  
Printer printer = new Printer(storage);  
  
counter.start();  
printer.start();  
}  
}
```

**Question:**

WAP to create a class MyThread derived from Thread class and override the run() method. Create a class ThreadDemo having a main method. Create 2 objects of MyThread and observe the behavior of threads.

```
class MyThread extends Thread {
```

```
public void run() {  
  
    for (int i = 1; i <= 5; i++) {  
  
        System.out.println(  
            Thread.currentThread().getName() +  
            " : " + i  
        );  
  
        try {  
            Thread.sleep(500);  
        } catch (InterruptedException e) {  
            System.out.println(e);  
        }  
    }  
}
```

```
    }  
  }  
}
```

```
public class ThreadDemo {
```

```
    public static void main(String[] args) {
```

```
        MyThread t1 = new MyThread();
```

```
        MyThread t2 = new MyThread();
```

```
        t1.setName("Thread 1");
```

```
        t2.setName("Thread 2");
```

```
        t1.start();
```

```
t2.start();
```

```
}
```

```
}
```